Київський національний університет імені Тараса Шевченка

Факультет інформаційних технологій

Кафедра програмних систем і технологій

МАГІСТЕРСЬКА КУРСОВА РОБОТА

Тема: “Методи та засоби штучного інтелекту для розпізнавання інформаційних фейків і протидії дезінформації в соціальних мережах”

Спеціальність – 121 “Інженерія програмного забезпечення”

ПОЯСНЮВАЛЬНА ЗАПИСКА

Студент
ІПЗм-12 _____ /Павло ЮРЧЕНКО/

Науковий керівник

к.т.н., доц. _____ /Тетяна КОВАЛЮК/

Консультант
з питань нормоконтролю

фахівець _____ /Марина ШАТИРКО/

Допускається до захисту
Завідувач кафедри

Д.т.н., проф. _____ /Олексій БИЧКОВ/

Київ – 2026

АНОТАЦІЯ

Магістерська курсова робота: 43 с., 4 рис., 14 табл., 4 дод., 30 джерел.

Тема роботи: розробка архітектури адаптивної інтелектуальної навчальної системи на основі великих мовних моделей для реалізації діалогового агента з функціями репетитора.

Об'єкт дослідження - процес адаптивного навчання користувача за допомогою діалогового агента в межах інтелектуальної навчальної системи.

Предмет дослідження - архітектурні моделі, методи адаптації великих мовних моделей, механізми пам'яті агента та програмні засоби реалізації мінімального практичного сценарію навчання базового JavaScript.

Мета роботи - спроєктувати узгоджену архітектуру AIHC із LLM-ядром і продемонструвати її працездатність на мінімальному практичному сценарії: генерація вправи, написання розв'язку, запуск тестів та надання адаптивного фідбеку.

У роботі систематизовано класичні компоненти ITS, проаналізовано сучасні підходи до використання LLM в освіті, запропоновано архітектуру агента-репетитора з моделлю стану студента, матрицею рішень і багаторівневою організацією пам'яті, а також реалізовано мінімальний браузерний прототип із локальною LLM, `Web Worker` для тестів та резервною логікою.

Реальні результати педагогічного експерименту у поточній версії відсутні; замість цього подано чесно оформлений дизайн експерименту та план оцінювання для подальшого дослідження.

Ключові слова: AIHC, ITS, велика мовна модель, LLM, діалоговий агент, репетитор, Ollama, Llama, JavaScript, адаптивний фідбек.

aScript education, Ollama, local LLM.

ЗМІСТ

СЛУЖБОВІ СТОРІНКИ / ЗАВДАННЯ.....2
АНОТАЦІЯ.....3
ABSTRACT.....4
СПИСОК УМОВНИХ ПОЗНАЧЕНЬ.....7
ВСТУП.....8
РОЗДІЛ 1. АНАЛІЗ ПРЕДМЕТНОЇ ГАЛУЗІ АДАПТИВНИХ ІНТЕЛЕКТУАЛЬНИХ НАВЧАЛЬНИХ СИСТЕМ.....10
1.1. Поняття та класифікація інтелектуальних навчальних систем (ITS/AIHC)10
1.2. Архітектурні компоненти AIHC.....10
1.3. Діалогові агенти у навчальних системах.....11
1.4. Огляд сучасних систем-репетиторів на основі ШІ.....11
1.5. Висновки до розділу 1.....12
РОЗДІЛ 2. ВЕЛИКІ МОВНІ МОДЕЛІ ЯК ОСНОВА ДІАЛОГОВОГО АГЕНТА..13
2.1. Архітектура трансформерів та принципи роботи LLM.....13
2.2. Порівняння LLM для навчальних застосунків.....13
2.3. Проєктування промптів, RAG і донавчання як методи адаптації.....14
2.4. Обмеження LLM: галюцинації, пам'ять, етика, приватність.....14
2.5. Обґрунтування вибору локальної відкритої Llama для прототипу.....14
РОЗДІЛ 3. ПРОЄКТУВАННЯ АРХІТЕКТУРИ AIHC.....16
3.1. Функціональні та нефункціональні вимоги.....16
3.2. Загальна архітектура системи.....17
3.3. Модель студента.....19
3.4. Модель знань і навчального контенту.....20
3.5. Модель і стратегії репетитора.....21
3.5.1. Формальне визначення агента-репетитора.....21
3.5.2. Архітектура агента - компонентна модель.....22

3.5.3. Матриця рішень і стратегія мінімальної підказки.....22
3.5.4. Типові сценарії взаємодії.....23
3.5.5. Цикл роботи агента: міркування -> дія -> спостереження.....23
3.5.6. Організація пам'яті агента.....24
3.5.7. Роль LLM і її обмеження.....25
РОЗДІЛ 4. РЕАЛІЗАЦІЯ ТА ЕКСПЕРИМЕНТАЛЬНА ПЕРЕВІРКА.....26
4.1. Вибір технологічного стеку.....26
4.2. Реалізація ключових модулів системи.....27
4.3. Проєктування промптів для генерації вправ і фідбеку.....30
4.4. Дизайн та проведення експерименту.....30
4.4.1. Функціональна валідація прототипу.....30
4.4.2. Дизайн педагогічного експерименту.....33
4.5. Аналіз результатів.....35
ВИСНОВКИ.....37
СПИСОК ВИКОРИСТАНИХ ДЖЕРЕЛ.....38
ДОДАТКИ.....42
Додаток А. Діаграми архітектури системи.....42
Додаток Б. Приклади промптів.....42
Додаток В. Приклади діалогів агента.....42
Додаток Г. Фрагменти програмного коду.....43

СПИСОК УМОВНИХ ПОЗНАЧЕНЬ

AIHC - адаптивна інтелектуальна навчальна система

ITS - Intelligent Tutoring System

LLM - Large Language Model

RAG - Retrieval-Augmented Generation

JSON - JavaScript Object Notation

API - Application Programming Interface

UI - User Interface

JS - JavaScript

Web Worker - ізольований потік браузера для клієнтського виконання коду

UserState - спрощена модель стану студента в прототипі

ВСТУП

Розвиток адаптивних інтелектуальних навчальних систем знову став одним із центральних напрямів освітніх технологій після появи великих мовних моделей. Якщо класичні ITS орієнтувалися переважно на явно задані правила, предметні моделі та обмежені діалогові інтерфейси, то LLM відкрили можливість будувати природніший діалог, оперативно генерувати навчальний контент і адаптувати зворотний зв'язок до поведінки конкретного студента [1], [6], [7], [8].

Для освітніх застосувань цього недостатньо розглядати лише якість текстової генерації. Ключовим стає архітектурне питання: які саме компоненти має містити система, як вони взаємодіють, де формується модель студента, як організовується пам'ять агента, яким чином ухвалюється педагогічна дія та як забезпечується безпечне виконання інструментальних операцій [2], [3], [9], [12].

Предметною областю цієї курсової роботи обрано навчання базового JavaScript. Такий вибір зручний з методичної та технічної точок зору: для невеликих програмістських вправ легко сформулювати тестовані критерії коректності, а процес взаємодії студента з репетитором природно вкладається у цикл «згенерувати задачу -> написати розв'язок -> запустити тести -> отримати адаптивний фідбек» [14], [15], [16], [17], [18].

Метою роботи є проєктування архітектури AIHC на основі великих мовних моделей для реалізації діалогового агента з функціями репетитора та демонстрація цієї архітектури на мінімальному, але реальному практичному сценарії прототипу.

Для досягнення поставленої мети необхідно розв'язати такі завдання:

1. проаналізувати класичні підходи до побудови ITS/AIHC та виділити їхні обов'язкові архітектурні компоненти;
2. узагальнити можливості й обмеження великих мовних моделей у навчальних сценаріях;
3. сформулювати функціональні та нефункціональні вимоги до системи;

4. запроєктувати модель студента, модель знань, стратегію репетитора, потоки даних і організацію пам'яті агента;
5. реалізувати мінімальний браузерний прототип для курсового практичного сценарію з локальною LLM та безпечним тестовим запуском коду;
6. запропонувати дизайн експериментальної перевірки без вигадування педагогічних результатів.

Об'єкт дослідження - процес адаптивної навчальної взаємодії між студентом і діалоговим агентом у межах інтелектуальної навчальної системи.

Предмет дослідження - архітектурні моделі, механізми пам'яті, політики прийняття рішень і програмні засоби побудови AIHC на основі LLM для сценарію репетиторського супроводу під час вивчення JavaScript.

У роботі використано методи системного аналізу, порівняльного огляду літератури, модульного проєктування програмних систем, прототипування, тестової функціональної валідації та формального опису агентної поведінки.

Наукова новизна роботи має стриманий характер і полягає не у створенні нової моделі навчання як такої, а у синтезі архітектурного підходу до AIHC, де LLM розглядається як один із компонентів ширшої системи з явною моделлю стану студента, політикою педагогічних дій, розмежуванням типів пам'яті та безпечним механізмом виконання програмних вправ.

Практична цінність полягає у створенні прототипу, який може бути розширений у повну магістерську кваліфікаційну роботу: додаванням довготривалої пам'яті, RAG-шару, реального контентного сховища, інструментів аналітики та повноцінного педагогічного експерименту.

РОЗДІЛ 1. АНАЛІЗ ПРЕДМЕТНОЇ ГАЛУЗІ АДАПТИВНИХ ІНТЕЛЕКТУАЛЬНИХ НАВЧАЛЬНИХ СИСТЕМ

1.1. Поняття та класифікація інтелектуальних навчальних систем (ITS/AIHC)

Інтелектуальна навчальна система традиційно розглядається як програмно-методичний комплекс, який намагається наблизити індивідуальне навчання до роботи з людським викладачем. На відміну від статичних e-learning платформ, ITS адаптує зміст, складність завдань, спосіб пояснення та форму зворотного зв'язку відповідно до поточного стану студента [1], [5], [6], [28].

Термін AIHC у межах цієї роботи використовується як сучасніше позначення тієї самої логіки, але з акцентом на адаптивність, агентність і поєднання класичних освітніх моделей з генеративними компонентами. Важливо, що навіть у LLM-орієнтованих системах адаптивність не виникає автоматично з самої моделі; вона вимагає явних архітектурних механізмів збору спостережень, зберігання стану та вибору педагогічних дій.

1.2. Архітектурні компоненти AIHC

Класична архітектура ITS спирається принаймні на чотири логічні підсистеми: модель предметної галузі, модель студента, педагогічну модель і інтерфейс взаємодії. У сучасних роботах цей поділ уточнюється, але не зникає: саме він дозволяє відокремити знання про предмет, знання про учня та знання про педагогічну стратегію [1], [2], [3], [4], [28].

Для цієї курсової роботи така декомпозиція принципова. Якщо одразу звести систему до одного LLM-виклику, втрачається можливість пояснити, де саме формується адаптивність. Тому в подальших розділах LLM розглядатиметься як генеративний шар, але не як повний замінник студентської моделі, контентної моделі та політики репетитора.

1.3. Діалогові агенти у навчальних системах

Натуральномовний діалог є природним інтерфейсом для навчання, оскільки дозволяє ставити уточнювальні запитання, пояснювати помилки та підтримувати мотивацію. Ранні системи з діалоговими можливостями були обмежені жорсткими сценаріями, тоді як нові LLM-агенти забезпечують значно вищу мовну гнучкість [14], [20], [22].

Особливо перспективним цей підхід є для навчання програмування. Тут агент може поєднати генерацію задач, аналіз спроб розв'язку, інтерпретацію результатів тестів і формування підказок без негайного розкриття повного коду, що узгоджується з підходами сучасних досліджень у programming education [14], [15], [16], [17], [18], [26], [27].

1.4. Огляд сучасних систем-репетиторів на основі ШІ

Сучасний ландшафт AI tutor systems містить як комерційні продукти, так і дослідницькі системи. Продуктові системи демонструють життєздатність формату, але зазвичай мало розкривають внутрішню архітектуру; академічні праці, навпаки, частіше дають детальний опис модулів і методів оцінювання [6], [7], [8], [19], [21], [23], [24].

Таблиця 1.1 - Порівняння сучасних систем-репетиторів на основі ШІ.

Система / напрям	**Фокус**	**Механізм адаптації**	**Релевантність для цієї роботи**
Khanmigo	Підтримка навчання та викладання	Діалоговий AI tutor і асистент викладача	Підходить як приклад продуктового AI tutor [23]
Duolingo Max	Мовне навчання	Roleplay, Explain My Answer, GPT-4-based UX	Корисний для аналізу стилю фідбеку [24]
ITS-CAL	Навчання програмування	Структуроване адаптивне середовище для задач для програмування	Дає приклад доменної адаптації [20]

Система / напрям	Фокус	Механізм адаптації	Релевантність для цієї роботи
LLM feedback studies	Програмування	Генерація коментарів до коду і педагогічних підказок	Показують сильні й слабкі сторони LLM у CS education [15], [17], [18], [26]

Для курсової роботи важливо не копіювати жодну з цих систем буквально, а виділити архітектурний інваріант: сучасний AI tutor має поєднувати мовну генерацію з контролем допомоги, явним поданням стану студента та механізмом перевірки зовнішніх дій.

1.5. Висновки до розділу 1

Аналіз предметної галузі показав, що архітектура AIHC не може зводитися до одного мовного модуля. Необхідними залишаються студентська модель, контентна модель, педагогічна політика, інтерфейс взаємодії та механізм спостереження за діями студента. Для сценарію навчання JavaScript найбільш логічною є архітектура, у якій LLM поєднується з інструментальним запуском тестів і адаптивною політикою репетитора [1], [2], [14], [15], [20], [22].

РОЗДІЛ 2. ВЕЛИКІ МОВНІ МОДЕЛІ ЯК ОСНОВА ДІАЛОГОВОГО АГЕНТА

2.1. Архітектура трансформерів та принципи роботи LLM

Великі мовні моделі базуються на архітектурі трансформера та навчаються передбачати наступні токени на великих корпусах тексту. На практиці це дає універсальний механізм, який можна використовувати для пояснення, формування підказок, перефразування й частково для планування дій, якщо система обмежує формат відповіді та подає моделі структурований контекст [7], [8].

У навчальних застосуваннях важливо відділяти дві ролі LLM: мовну й керувальну. Мовна роль полягає у створенні зрозумілих відповідей, а керувальна - у виборі, що саме треба сказати в даному педагогічному контексті. Друга роль не повинна повністю покладатися на статистичну генерацію; вона потребує додаткової політики та пам'яті системи.

2.2. Порівняння LLM для навчальних застосунків

Для освітніх систем доступні як закриті провідні моделі, так і відкриті локальні моделі. Закриті сервіси зазвичай демонструють вищу якість міркування й пояснень, однак створюють залежність від зовнішнього API, підвищують ризики для приватності й ускладнюють відтворюваність. Відкриті локальні моделі є слабшими в середньому, але дають більше контролю над витратами, обробкою даних і правилами інтеграції [7], [8], [10], [17], [18], [19], [29].

Для задачі базового JavaScript-репетитора не потрібна максимальна генеративна потужність, натомість важливі локальність, передбачуваність і можливість працювати з невеликим структурованим JSON-контрактом. Саме тому в роботі обрано локальне середовище виконання `Ollama + llama3.1:8b` як цільове рішення для прототипу [25], [30].

2.3. Проєктування промптів, RAG і донавчання як методи адаптації

На практиці адаптація LLM до навчального домену може здійснюватися щонайменше трьома способами: через проєктування промптів, через генерацію з доповненим пошуком (retrieval-augmented generation, RAG) та через донавчання. Проєктування промптів є найдоступнішим шляхом для курсового проєкту, оскільки дозволяє нав'язати формат JSON, заборону на повне розкриття розв'язку і стиль репетитора без окремого етапу донавчання [11], [12], [25].

RAG є логічним наступним кроком для розвитку системи, адже дозволяє під'єднати зовнішнє сховище навчального контенту, прикладів і теоретичних пояснень. Донавчання у межах поточної курсової не реалізується через обмеження масштабу, але в архітектурному описі воно розглядається як потенційний засіб стабілізації стилю та якості педагогічного фідбеку.

2.4. Обмеження LLM: галюцинації, пам'ять, етика, приватність

Освітнє застосування LLM пов'язане з ризиками галюцинацій, надмірної допомоги, відсутності стабільної довготривалої пам'яті та з етичними питаннями щодо прозорості, приватності й залежності студента від системи [9], [10], [13], [29]. Для репетиторського сценарію ці ризики є особливо критичними: неправильно сформульована підказка може не просто помилково інформувати, а й зруйнувати навчальний ефект завдяки передчасному розкриттю відповіді.

Звідси випливають архітектурні наслідки для курсової роботи: необхідність контролю формату відповіді, повторних спроб із переходом до резервного режиму при невалідному JSON, окремого механізму політики репетитора, обмеження на розкриття розв'язку та безпечного виконання лише тих інструментальних дій, які не вимагають виконання коду на сервері.

2.5. Обґрунтування вибору локальної відкритої Llama для прототипу

Вибір локальної Llama через Ollama зумовлено кількома факторами. По-перше, це відповідає вимозі використовувати безкоштовне локальне середовище виконання

без звернення до платних хмарних API. По-друге, Ollama надає достатньо простий HTTP API для інтеграції у полегшений прототип. По-третє, модель класу 8B достатня для генерації невеликих задач, коротких підказок та структурованих відповідей у межах базового навчального сценарію [17], [18], [19], [25], [30].

Водночас у роботі прямо визнається, що такий вибір є компромісом: локальна модель може поступатися сильнішим закритим системам за точністю та стабільністю. Саме тому в архітектурі одразу передбачено нормалізацію відповіді, резервні вправи та шаблонний фідбек на випадок, коли LLM недоступна або повертає некоректні дані.

Отже, LLM у межах цієї роботи виступає не як автономний педагог, а як гнучкий генеративний компонент, вбудований у ширшу архітектуру AIHC з явною моделлю стану, механізмом пам'яті та політикою ухвалення рішень [7], [9], [12], [25], [30].

РОЗДІЛ 3. ПРОЄКТУВАННЯ АРХІТЕКТУРИ АІНС

3.1. Функціональні та нефункціональні вимоги

Система проєктується не як універсальний освітній портал, а як мінімальний практичний сценарій для репетиторського супроводу під час вивчення базового JavaScript. Це дозволяє сформулювати чіткі вимоги й водночас залишити місце для подальшого масштабування.

Таблиця 3.1 - Функціональні вимоги до прототипу АІНС.

Код	Функціональна вимога	Статус у прототипі
FR1	Згенерувати невелику вправу з JavaScript у структурованому форматі	реалізовано
FR2	Дозволити студенту написати або відредагувати розв'язок	реалізовано
FR3	Запустити тести лише на клієнті, у Web Worker з таймаутом	реалізовано
FR4	Надати адаптивний фідбек без негайного повного розв'язку	реалізовано
FR5	Підтримати мінімальну модель стану студента в межах сесії	реалізовано
FR6	Обробити невалідний JSON від моделі через retry/fallback	реалізовано

Таблиця 3.2 - Нефункціональні вимоги до прототипу АІНС.

Код	Нефункціональна вимога	Примітка
NFR1	Використовувати локальний open-source runtime	цільовий runtime: Ollama + llama3.1:8b
NFR2	Не виконувати user code на сервері	виконано
NFR3	Забезпечити модульну архітектуру з явним поділом відповідальностей	виконано
NFR4	Забезпечити пояснювану логіку педагогічного рішення	виконано через TutorPolicy
NFR5	Мати деградаційний режим без LLM	виконано
NFR6	Не вигадувати результати експериментів	виконано в документі

3.2. Загальна архітектура системи

Архітектура пропонованої AIHC складається з п'яти логічних шарів: інтерфейсного, шару сприйняття та інтерпретації запиту, шару педагогічного міркування, шару інструментальних дій і шару пам'яті. Центральним елементом виступає агент-репетитор, який не просто генерує текст, а працює в циклі спостереження, оновлення стану й вибору наступної дії [1], [11], [12].

На практичному рівні прототип реалізує лише частину цієї архітектури, але вона вже достатня, щоб продемонструвати головний принцип: генерація вправи й текстовий фідбек мають бути відокремлені від виконання студентського коду та від моделі стану користувача. Таке розділення зменшує ризик змішування педагогічної логіки та інструментального рантайму.

Таблиця 3.3 - Компоненти запропонованої архітектури AIHC.

Компонент	Призначення	Статус
Chat UI	Інтерфейс студента, показ вправи, коду, результатів тестів і фідбеку	реалізовано
Perception Module	Класифікація наміру й нормалізація запиту	описано концептуально
SessionMemory	Поточний стан, transcript tail, історія помилок	реалізовано
Summary Memory	Стисле зведення попередніх сесій	описано концептуально
Vector/Profile Memory	Довготривалий профіль студента між сесіями	описано концептуально
Knowledge Memory	Контентна база, RAG, концепти, пояснення	описано концептуально
TutorPolicy / Reasoning Module	Вибір педагогічної дії	реалізовано у rule-based вигляді
ExerciseGenerator	Створення вправи через LLM або fallback	реалізовано
ClientTestRunner	Запуск тестів у Web Worker	реалізовано
FeedbackEvaluator	Формування адаптивного фідбеку	реалізовано
Generation Module / LLM	Генерація структурованих exercise/feedback JSON	реалізовано частково через Ollama API

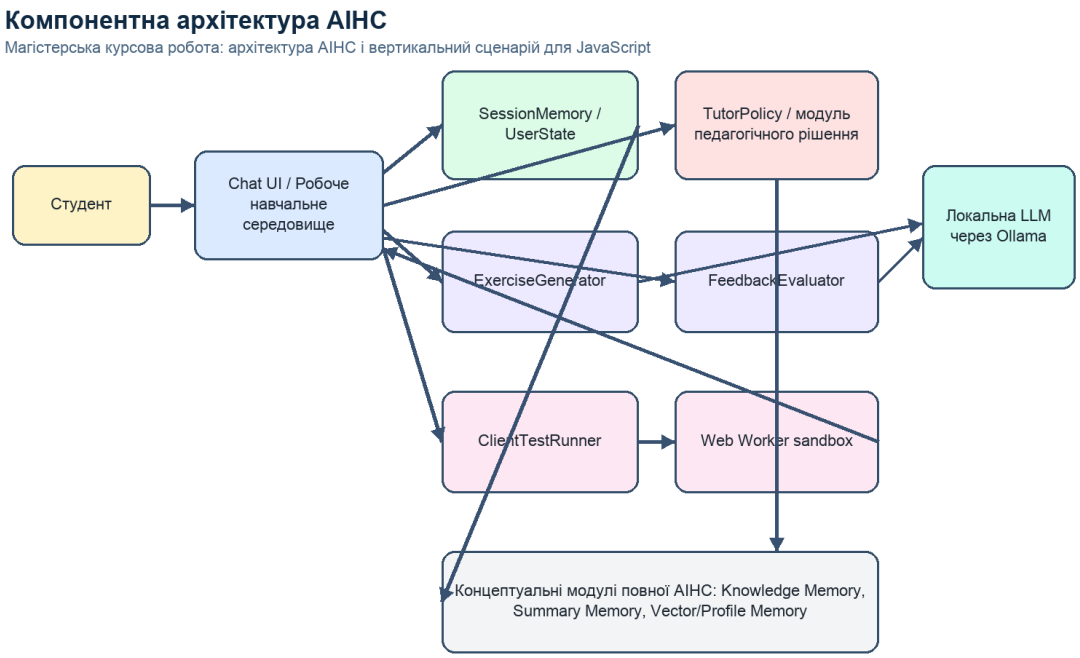

Рисунок 3.1 - Компонентна архітектура AIHC для сценарію навчання базового JavaScript.

На рисунку 3.1 узагальнено взаємодію концептуальних і реалізованих модулів AIHC. До практичної частини увійшли лише ті компоненти, які реально присутні в репозиторії.

3.3. Модель студента

У центрі персоналізації перебуває модель студента. Повноцінні ITS використовують для цього складніші probabilistic або knowledge tracing механізми, але для мінімального практичного сценарію доцільно застосувати спрощений вектор стану `UserState`, який усе ж зберігає ключові сигнали для педагогічного рішення [2], [3], [4].

Таблиця 3.4 - Поля спрощеної моделі стану студента `UserState`.

Поле UserState	Призначення
knowledgeLevel	Груба оцінка поточного рівня знань студента
errorHistory	Список останніх помилок або типів збоїв
confidence	Суб'єктивно-операційна оцінка впевненості системи у стані студента
currentTopic	Поточна навчальна тема

Поле UserState	Призначення
attemptsCount	Кількість спроб для поточної вправи
lastAction	Остання педагогічна дія агента

Оновлення `UserState` відбувається після кожної значущої взаємодії. Результати тестів, тип першої помилки, повторення того самого збою та запит на пояснення інтерпретуються як сигнали для оновлення впевненості, історії помилок і вибору наступної дії. У повній системі до цього вектора могли б додаватися concept-level mastery, preferred explanation style та довготривалий профіль.

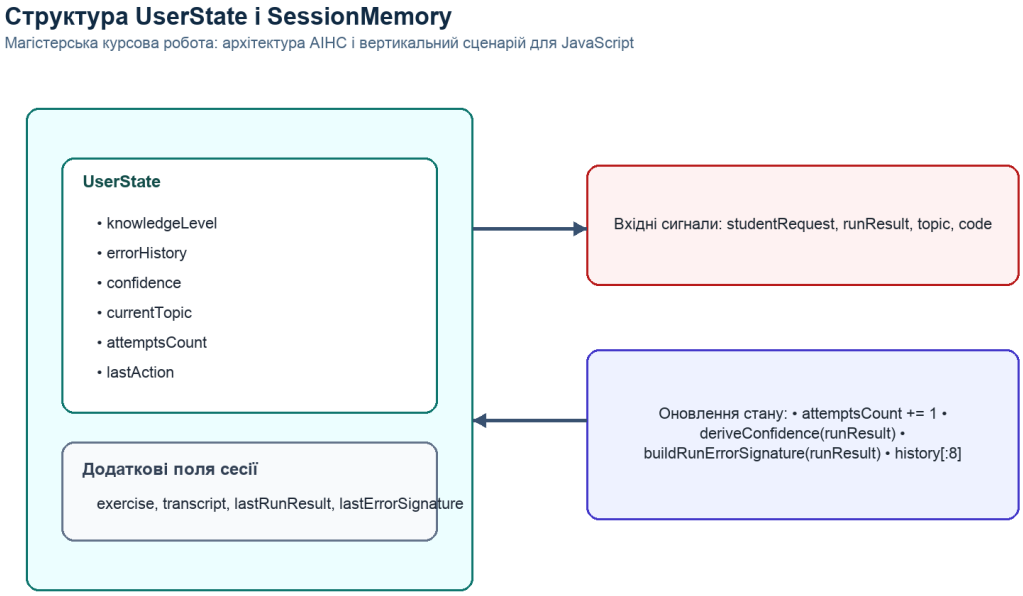

Рисунок 3.2 - Структура `UserState` та зв'язок із `SessionMemory`.

3.4. Модель знань і навчального контенту

Модель знань у повній AIHC повинна поєднувати концепти предметної галузі, навчальні цілі, типові помилки та набір контентних артефактів: вправи, тести, пояснення, контрольні запитання. У класичному ITS це часто реалізується через експліцитну предметну модель, а в сучасних LLM-based системах може бути доповнене retrieval-шаром та векторним індексом [3], [11].

У прототипі ця модель навмисно спрощена до JSON-репрезентації вправи з полями `title`, `prompt`, `starterCode`, `functionName`, `tests`, `concepts`, `rubric`. Таке рішення дозволяє зберегти місце для майбутнього RAG, але не розширювати практичну частину понад необхідний мінімум.

Архітектурно knowledge memory поділяється на чотири типи сутностей: концепти JavaScript, шаблони задач, перевірочні тести та пояснювальні матеріали. У повній версії системи ці сутності можуть бути пов'язані у вигляді графа концептів і retrieval-індексу; в поточній версії їх роль частково виконує вбудований резервний набір вправ і prompt-контракт для LLM.

3.5. Модель і стратегії репетитора

3.5.1. Формальне визначення агента-репетитора

Діалоговий агент у цій роботі визначається як керована система, що підтримує навчальний діалог і виконує педагогічно осмислені дії, спираючись на спостереження за студентом. На відміну від звичайного чат-бота, агент-репетитор пов'язує мовну відповідь із контекстом навчальної задачі, поточним станом користувача та інструментальними результатами перевірки розв'язку.

$S = K \times E \times C \times Topic \times Attempts \times M$

$A = \{generate_exercise, give_minimal_hint, give_targeted_hint, explain_concept, celebrate_and_reflect, request_retry, escalate\}$

$T: S \times O \to S$

$P: S \times O \to A$

$R: S \times A \times Ctx \to Text$

Тут `K` позначає оцінку знань, `E` - історію помилок, `C` - впевненість, `Topic` - поточну тему, `Attempts` - кількість спроб, `M` - пам'ять сесії, `O` - нові спостереження, `A` - множину дій агента, `T` - функцію переходу стану, `P` - політику, а `R` - функцію генерації відповіді. Такий опис дозволяє відокремити логіку міркування агента від мовної генерації [11], [12].

3.5.2. Архітектура агента - компонентна модель

Компонентна модель агента складається з таких блоків. Модуль сприйняття інтерпретує вхідний запит і визначає режим взаємодії. Модуль пам'яті акумулює поточний контекст і довготривалі відомості. Модуль міркування оцінює стан студента та обирає педагогічну дію. Модуль дій викликає генерацію вправи, перевірку тестів або пояснення. Модуль генерації формує кінцеву текстову відповідь на основі політики та контексту.

У прототипі ця декомпозиція реалізована не повністю: роль модуля сприйняття виконують явний режим `interactionMode: "explain"` для кнопки пояснення та прості текстові евристики для вільного запиту, а роль модуля генерації виконує локальна LLM або резервний механізм. Однак саме така схема використовується як основа повнішої системи в магістерському проєкті.

3.5.3. Матриця рішень і стратегія мінімальної підказки

Ключова вимога до агента-репетитора полягає в тому, щоб допомога була мінімально достатньою. Після технічного аудиту правило ескалації було уточнено: перша невдала спроба дає `minimal_hint`, а `targeted_hint` з'являється лише після повторення тієї самої помилки або серії кількох невдалих спроб.

Таблиця 3.5 - Матриця педагогічних дій агента-репетитора.

Стан користувача	**Дія агента**	**Тип відповіді**
Усі тести пройдено	celebrate_and_reflect	Підкріпити успіх і запропонувати коротку рефлексію
Перша помилка	minimal_hint	Натякнути на ідею без готового розв'язку
Повторна помилка	targeted_hint	Вказати на конкретний збій і пов'язаний концепт
Синтаксична помилка	syntax_repair	Підказати зону пошуку помилки
Запит на пояснення	concept_explanation	Дати коротке

Стан користувача	Дія агента	Тип відповіді
		концептуальне пояснення без повного коду

У коді ця матриця реалізована в модулі `TutorPolicy` через набір явних правил. Для більш складної системи вона може бути замінена комбінацією рушія правил, адаптивної стратегії типу multi-armed bandit або навчання політики, але для курсової роботи важливо насамперед показати сам принцип явного розділення педагогічної дії та генеративної реалізації.

3.5.4. Типові сценарії взаємодії

- Правильна відповідь: агент фіксує проходження тестів, підсилює впевненість студента та пропонує коротке пояснення власними словами.
- Помилка: агент дає мінімальний натяк, орієнтований на найперший збій або простий приклад.
- Повторна помилка: агент переходить до більш адресної підказки (`targeted_hint`) і конкретніше називає, який фрагмент логіки слід перевірити.
- Запит на пояснення: агент переходить у режим концептуального пояснення (`concept_explanation`) і дає теоретичне пояснення без завершеного розв'язку.

3.5.5. Цикл роботи агента: міркування -> дія -> спостереження

Робочий цикл агента описується схемою «міркування -> дія -> спостереження», яка відповідає підходу ReAct. На етапі міркування система інтерпретує нові спостереження й оновлює `UserState`. На етапі дії обирається педагогічна дія: створити вправу, дати підказку, пояснити концепт або запросити нову спробу. На етапі спостереження система отримує нові результати, зокрема результати тестів, і повторює цикл [12].

Рисунок 3.3 - Діаграма послідовності для сценарію «згенерувати вправу -> пройти тести -> дати фідбек».

3.5.6. Організація пам'яті агента

Таблиця 3.6 - Організація пам'яті агента.

Тип пам'яті	Призначення	Статус
Buffer / Session Memory	Контекст поточної сесії, останні повідомлення, поточна вправа, результат тестів	реалізовано
Summary Memory	Стислий підсумок попереднього діалогу, придатний для наступної сесії	концептуально
Vector / Profile Memory	Стійкий профіль студента між сесіями, embeddings і патерни труднощів	концептуально
Knowledge Memory	Навчальний контент, концепти, приклади, RAG-	концептуально

Тип пам'яті	Призначення	Статус
	корпус	

Такий поділ потрібен для того, щоб не змішувати короткочасний контекст із довготривалою персоналізацією та із зовнішнім предметним знанням. Саме це є однією з ключових архітектурних тез роботи: пам'ять агента не повинна ототожнюватися з контекстним вікном LLM.

3.5.7. Роль LLM і її обмеження

У пропонованій AIHC LLM виконує роль ядра модуля генерації, але не є єдиним носієм логіки системи. Вона перетворює структурований контекст у зрозумілу підказку або вправу, однак сама політика, модель стану та безпечне середовище виконання мають бути винесені за межі моделі. Такий підхід дозволяє зменшити вплив галюцинацій і краще контролювати освітню доцільність відповіді [9], [10], [13].

Таким чином, найбільш цінним результатом цього розділу є не конкретний інструмент або промпт, а архітектурна схема, яка пояснює місце кожного модуля в системі та відразу розділяє концептуальний рівень AIHC і той обмежений набір компонентів, який реально реалізовано в прототипі.

РОЗДІЛ 4. РЕАЛІЗАЦІЯ ТА ЕКСПЕРИМЕНТАЛЬНА ПЕРЕВІРКА

4.1. Вибір технологічного стеку

Оскільки в репозиторії на початку роботи не було прикладної реалізації, практичну частину свідомо обмежено до одного мінімального практичного сценарію. Метою було не створити повну платформу, а отримати реальний кодовий артефакт, що демонструє архітектурні рішення з розділу 3.

Таблиця 4.1 - Технологічний стек прототипу.

Шар	Технологія	Обґрунтування
Браузерний інтерфейс	HTML/CSS/JavaScript ES modules	мінімальний браузерний прототип без окремого фреймворку
Середовище LLM	Ollama + llama3.1:8b	локальне відкрите середовище виконання відповідно до вимог роботи
Стан сесії	SessionMemory у пам'яті процесу	простий і прозорий варіант для мінімального практичного сценарію та майбутнього JSON-експорту сесій
Запуск тестів	Web Worker + timeout	без виконання коду на сервері та з ізоляцією обчислень
Інженерні тести	node --test	перевірка чистої логіки модулів

У поточному середовищі локальний виконуваний файл `ollama` не був встановлений, тому прототип додатково отримав резервний режим. Це важливий практичний висновок: для дослідницької системи потрібно з самого початку передбачати резервну логіку, а не вважати зовнішнє середовище виконання гарантовано доступним [25].

На рисунку 4.1 показано, як у прототипі розведено браузерний інтерфейс, локальний LLM-виклик, резервний режим і клієнтський запуск тестів у `Web Worker`.

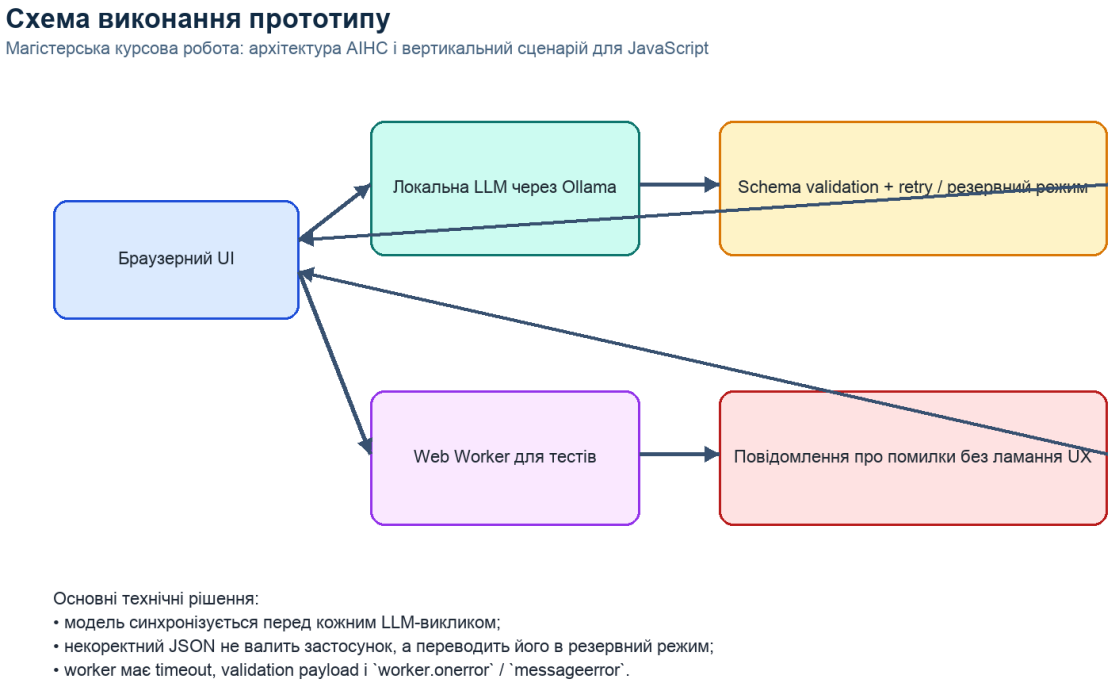

Рисунок 4.1 - Схема виконання прототипу з локальною LLM, резервним режимом і `Web Worker`.

4.2. Реалізація ключових модулів системи

У практичній частині описуються лише ті модулі, які реально присутні в коді репозиторію. Повна архітектура AIHC ширша, але в цьому підрозділі фіксується тільки фактичний обсяг реалізації.

Таблиця 4.2 - Реалізовані модулі прототипу.

Модуль	**Файли**	**Функція**
Інтерфейс користувача	`prototype/ index.html`, `prototype/src/main.js`	Інтерфейс вибору теми, редактор коду, журнал взаємодій, синхронізація моделі та дружні повідомлення про помилки.
SessionMemory	`prototype/src/state/ sessionMemory.js`	Зберігає `UserState`, поточну вправу, історію повідомлень, першу зафіксовану помилку та формує полегшений JSON-експорт сесії.
TutorPolicy	`prototype/src/ services/tutorPolicy.js`	Політика на основі правил з коректною ескалацією: перша помилка -> `minimal_hint`, повторна та сама -> `targeted_hint`, а кнопка пояснення через `interactionMode: "explain"` завжди веде до `concept_explanation`.
ExerciseGenerator	`prototype/src/ services/ exerciseGenerator.js`	Генерація вправи через LLM, явна перевірка структури та перехід до резервної вправи при

Модуль	Файли	Функція
		невідповідному JSON.
ClientTestRunner	`prototype/src/ services/ clientTestRunner.js`, `prototype/src/workers/jsSandboxWorker.js`	Клієнтський запуск тестів у `Web Worker` з тайм-аутом, перевіркою вхідних даних, `worker.onerror` і `messageerror`.
FeedbackEvaluator	`prototype/src/ services/ feedbackEvaluator.js`	Адаптивний фідбек через LLM або резервний режим; резервне пояснення формує змістовне пояснення концепту.
OllamaClient	`prototype/src/ services/ollamaClient.js`	HTTP API, перевірка доступності, перевірка структури та повторні спроби з переходом до резервного режиму для JSON-відповідей.

Такий обсяг реалізації відповідає принципу пріоритету архітектури: замість великої кількості другорядних функцій свідомо реалізовано лише ті модулі, які демонструють зв'язок між генерацією задачі, моделлю стану, тестовим запуском, режимом пояснення та адаптивним репетиторським фідбеком. На рівні `TutorPolicy` і `main.js` додано явний режим пояснення, тому натискання кнопки `Пояснити тему` завжди переводить агента до `concept_explanation`, а на рівні `SessionMemory` та UI реалізовано полегшений JSON-експорт поточної сесії як допоміжний інструмент для подальших досліджень типу A/B або кросовер-досліджень.

4.3. Проєктування промптів для генерації вправ і фідбеку

Для прототипу сформовано дві основні стратегії формування промптів: генерація вправи та формування адаптивного фідбеку. В обох випадках система вимагає лише JSON-відповідь, що спрощує валідацію й нормалізацію. Такий підхід узгоджується з рекомендаціями щодо керованого використання LLM у навчанні та програмуванні [9], [12], [15], [16], [17], [18], [26], [27].

- стиль відповіді має бути репетиторським, коротким і підтримувальним;
- повний розв'язок не повинен видаватися одразу;
- після повторних невдалих спроб дозволяється більш конкретна, але все ще не повна підказка;
- exercise JSON повинен містити сигнатуру функції, список тестів і рубрику;
- якщо модель повертає невалідний JSON або недоступна, система переходить у резервний режим.

Таким чином, проєктування промптів у цій роботі відіграє роль засобу обмеження та структуризації генерації, а не самодостатнього механізму педагогічної адаптації.

4.4. Дизайн та проведення експерименту

4.4.1. Функціональна валідація прототипу

У поточній роботі було виконано лише ту функціональну перевірку, яку можна підтвердити реальними артефактами середовища. Освітні результати, UX-оцінки та ефективність навчання тут не вигадуються. Натомість модульними тестами й швидкою контрольною перевіркою підтверджено коректну ескалацію підказок, явний режим пояснення, полегшений JSON-експорт сесії та базову стійкість прототипу до невалідних LLM-відповідей.

Таблиця 4.3 - Результати функціональної валідації прототипу.

Об'єкт перевірки	**Артефакт**	**Результат**	**Коментар**
Політика підказок	`npm test`	пройдено	Перевірено: перша помилка -> `minimal_hint`; повторна та сама помилка -> `targeted_hint`.
Режим пояснення	`npm test`, `npm run check`	пройдено	Підтверджено, що явний `interactionMode: "explain"` гарантує перехід до `concept_explanation`, а пояснення лишається змістовним.
Валідація структури JSON	`npm test`	пройдено	Перевірено `null`, масив, рядок і частковий об'єкт для exercise/feedback JSON.
Sandbox worker	`npm test`	пройдено	Перевірено `missing function`, timeout path, `functionName`

Об'єкт перевірки	Артефакт	Результат	Коментар
			validation, `worker` error handling.
Експорт сесії	`npm test`, `npm run check`	пройдено	Перевірено поля `topic`, `difficulty`, `attemptsCount`, `firstFailure`, `lastAction`, `transcript`, `finalRunStatus`, `timestamp`.
Швидка контрольна перевірка	`npm run check`	пройдено	Скрипт без важкого стеку швидко перевіряє потік пояснення та формування JSON-експорту сесії.
Повний сценарій з локальною Ollama	ручний запуск у браузері	потребує ручної перевірки	Потребує встановленого `ollama` і локальної моделі `llama3.1:8b`.

Отже, інженерна функціональність мінімального практичного сценарію підтверджена для чистих модулів, явного режиму пояснення, JSON-експорту сесії та

базової статичної доставки; повний інтерактивний сценарій з локальним Ollama має бути додатково перевірений вручну у браузері.

4.4.2. Дизайн педагогічного експерименту

Реальний педагогічний експеримент у межах цієї курсової роботи ще не проводився, тому тут подається лише чесно оформлений дизайн дослідження, який може бути використаний на наступному етапі [14], [19], [21], [22], [26], [27]. JSON-експорт сесії розглядається як допоміжний інструмент для накопичення структурованих логів у майбутньому A/B- або кросовер-дослідженні.

Найдоцільнішим для невеликої вибірки видається кросовер-дизайн. Одна група спочатку працює з AIHC-прототипом, друга - з контрольним сценарієм без адаптивного репетиторського фідбеку; після короткого періоду вирівнювання умови міняються місцями.

Таблиця 4.4 - Гіпотези педагогічного експерименту.

Гіпотеза	Зміст
H1	Адаптивний агент покращує результат підсумкового тесту порівняно з режимом без адаптивного фідбеку.
H2	Адаптивний агент скорочує кількість безрезультатних спроб і час до коректного розв'язку.
H3	Адаптивний агент підвищує суб'єктивне відчуття підтримки без зростання залежності від готових відповідей.

Таблиця 4.5 - План параметрів та метрик педагогічного експерименту.

Параметр	План
Дизайн	crossover або A/B із попереднім і

Параметр	План
	підсумковим тестом; обидві умови проходять одну й ту саму тематичну міні-сесію.
Цільова вибірка	24-40 студентів, які вивчають базові теми JavaScript.
Основні метрики	правильність розв'язку, час виконання, кількість спроб, якість самостійного пояснення.
Додаткові метрики	самооцінка впевненості, суб'єктивна корисність підказок, потреба в повторному поясненні.
Статистичний аналіз	paired t-test або Wilcoxon; effect size розраховувати лише після отримання реальних даних.

Таблиця 4.6 - Операціоналізація змінних педагогічного експерименту.

Тип змінної	Позначення	Опис
Незалежна	X	Режим навчання: адаптивний агент або контрольний сценарій без адаптивного фідбеку.
Залежна	Y1	Результат підсумкового тесту з базового JavaScript.
Залежна	Y2	Час до коректного розв'язку.
Залежна	Y3	Кількість спроб до першого коректного розв'язку.
Контрольована	C1	Однакова тема,

Тип змінної	Позначення	Опис
		складність вправи, набір тестів і часовий ліміт сесії.

Таблиця 4.7 - Шаблон таблиці для ручного занесення результатів експерименту.

Метрика	Умова A	Умова B	Примітка
Середній pre-test	[ПОТРІБНО РУЧНЕ ВТРУЧАННЯ]	[ПОТРІБНО РУЧНЕ ВТРУЧАННЯ]	
Середній post-test	[ПОТРІБНО РУЧНЕ ВТРУЧАННЯ]	[ПОТРІБНО РУЧНЕ ВТРУЧАННЯ]	
Середній час до розв'язку	[ПОТРІБНО РУЧНЕ ВТРУЧАННЯ]	[ПОТРІБНО РУЧНЕ ВТРУЧАННЯ]	
Середня кількість спроб	[ПОТРІБНО РУЧНЕ ВТРУЧАННЯ]	[ПОТРІБНО РУЧНЕ ВТРУЧАННЯ]	
Статистичний висновок	[ПОТРІБНО РУЧНЕ ВТРУЧАННЯ]	[ПОТРІБНО РУЧНЕ ВТРУЧАННЯ]	Заповнюється після розрахунків.

Для уникнення спекулятивних висновків у роботі фіксується лише план статистичного аналізу: перевірка нормальності розподілу, вибір параметричного або непараметричного критерію, оцінка величини ефекту та окремий аналіз перенесення знань на delayed test. Числові результати, effect size та p-values мають бути внесені лише після збору реальних даних.

[ПОТРІБНО РУЧНЕ ВТРУЧАННЯ: після проведення реального педагогічного експерименту додати вибірку, статистичні результати, величини ефекту та інтерпретацію.]

4.5. Аналіз результатів

На момент завершення автоматично згенерованої чернетки наявні лише результати інженерної валідації. Їх достатньо, щоб стверджувати: у репозиторії присутній реальний мінімальний практичний сценарій, який структурно відповідає описаній архітектурі, але вони недостатні для висновків про педагогічну ефективність.

Практичний результат полягає в тому, що документ і код узгоджено. У тексті не заявляються модулі, яких немає у реалізації, а в коді реалізовано саме ті компоненти, які описано в підрозділах 4.1-4.3: інтерфейс користувача, `SessionMemory` з JSON-експортом, `TutorPolicy` з явним режимом пояснення, `ExerciseGenerator`, `ClientTestRunner`, `FeedbackEvaluator` та `OllamaClient`.

Ключове обмеження полягає в тому, що локальне середовище виконання Ollama не було встановлене у поточному середовищі. Це означає, що частина практичної логіки перевірялася у резервному режимі, а повна робота з реальною локальною моделлю потребує ручного запуску у браузері.

Подальше розширення розділу 4 можливе лише після збору реальних освітніх даних у межах A/B- або кросовер-експерименту; до цього моменту аналіз результатів має залишатися описом інженерної валідації та плану педагогічного дослідження.

ВИСНОВКИ

У роботі показано, що для побудови діалогового агента з функціями репетитора недостатньо розглядати лише LLM; потрібна повна архітектура AIHC із моделлю студента, моделлю знань, політикою репетитора, організацією пам'яті та механізмом інструментальних дій.

Найважливішим результатом є архітектурна схема агента, в якій виокремлено модуль сприйняття, модуль пам'яті, модуль міркування, модуль дій і модуль генерації, а також формально описано стан, дії, політику й функцію переходу.

Для практичної частини реалізовано мінімальний практичний сценарій репетиторського супроводу з JavaScript: генерація вправи, введення коду, запуск тестів у `Web Worker`, адаптивний фідбек, `SessionMemory`, явний режим пояснення та резервна логіка у випадку недоступності або нестабільності локальної LLM.

Документ побудовано чесно щодо емпіричної частини: реальні освітні результати не вигадувалися, а замість них подано інженерну функціональну валідацію та дизайн майбутнього педагогічного експерименту.

Отриманий прототип і текстова чернетка можуть слугувати базою для подальшого розширення в повну магістерську роботу з додаванням довготривалої пам'яті, RAG, багатшої моделі навчального контенту та реального оцінювання впливу системи на навчальні результати.

СПИСОК ВИКОРИСТАНИХ ДЖЕРЕЛ

[1] Alkhatlan A., Kalita J. Intelligent Tutoring Systems: A Comprehensive Historical Survey with Recent Developments. International Journal of Computer Applications. 2019. DOI: 10.5120/ijca2019918451.

[2] Conati C., Kardan S. Student Modeling: Supporting Personalized Instruction, from Problem Solving to Exploratory Open-Ended Activities. AI Magazine. 2013. DOI: 10.1609/aimag.v34i3.2483.

[3] Abdelrahman G., Wang Q., Nunes B. P. et al. Knowledge Tracing: A Survey. ACM Computing Surveys. 2023. DOI: 10.1145/3569576.

[4] Corbett A. T., Anderson J. R. Knowledge tracing: Modeling the acquisition of procedural knowledge. User Modeling and User-Adapted Interaction. 1995. DOI: 10.1007/BF01099821.

[5] Dai C. P., Ke F. et al. Educational applications of artificial intelligence in simulation-based learning: A systematic mapping review. Computers and Education: Artificial Intelligence. 2022. DOI: 10.1016/j.caeai.2022.100087.

[6] Letourneau K., Deslandes Martineau M., Charland P., Karran A. A systematic review of AI-driven intelligent tutoring systems (ITS) in K-12 education. npj Science of Learning. 2025. DOI: 10.1038/s41539-025-00320-7.

[7] Xu H., Gan W., Qi Z., Wu J., Yu P. S. Large Language Models for Education: A Survey. arXiv preprint arXiv:2405.13001. 2024. URL: https://arxiv.org/abs/2405.13001.

[8] Wang S., Xu T., Li H., Zhang C., Liang J., Tang J. Large Language Models for Education: A Survey and Outlook. IEEE Signal Processing Magazine. 2025. DOI: 10.1109/MSP.2025.3594309.

[9] Yan L., Sha L., Zhao L., Li X. Practical and ethical challenges of large language models in education: A systematic scoping review. British Journal of Educational Technology. 2024. DOI: 10.1111/bjet.13370.

[10] Shi Y., Yu Z., Dong Y., Chen Y. Large language models in education: a systematic review of empirical applications, benefits, and challenges. Computers and Education: Artificial Intelligence. 2026. DOI: 10.1016/j.caeai.2025.100529.

[11] Lewis P., Perez E., Piktus A., Petroni F. et al. Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. arXiv preprint arXiv:2005.11401. 2020. URL: https://arxiv.org/abs/2005.11401.

[12] Yao S., Zhao J., Yu D., Du N., Shafran I., Narasimhan K. ReAct: Synergizing Reasoning and Acting in Language Models. arXiv preprint arXiv:2210.03629. 2022. URL: https://arxiv.org/abs/2210.03629.

[13] Huang L., Yu W., Ma W., Zhong W., Feng Z., Wang H. A Survey on Hallucination in Large Language Models: Principles, Taxonomy, Challenges, and Open Questions. arXiv preprint arXiv:2311.05232. 2023. URL: https://arxiv.org/abs/2311.05232.

[14] Crow T., Luxton-Reilly A., Wuensche B. Intelligent tutoring systems for programming education. Proceedings of the 20th Australasian Computing Education Conference. 2018. DOI: 10.1145/3160489.3160492.

[15] Jacobs C., Jaschke S. Evaluating the Application of Large Language Models to Generate Feedback in Programming Education. 2024 IEEE Global Engineering Education Conference (EDUCON). 2024. DOI: 10.1109/EDUCON60312.2024.10578838.

[16] Jin I., Lee M., Shin H., Kim J. Teach AI How to Code: Using Large Language Models as Teachable Agents for Programming Education. Proceedings of the CHI Conference on Human Factors in Computing Systems. 2024. DOI: 10.1145/3613904.3642349.

[17] Koutcheme C., Dainese N., Sarsa S., Hellas A. Open Source Language Models Can Provide Feedback: Evaluating LLMs' Ability to Help Students Using GPT-4-As-A-Judge. Proceedings of the 2024 on Innovation and Technology in Computer Science Education. 2024. DOI: 10.1145/3649217.3653612.

[18] Koutcheme C., Dainese N., Sarsa S., Hellas A. Evaluating Language Models for Generating and Judging Programming Feedback. Proceedings of the 56th ACM Technical Symposium on Computer Science Education. 2025. DOI: 10.1145/3641554.3701791.

[19] Raihan A., Siddiq F., Santos A., Zampieri M. Large Language Models in Computer Science Education: A Systematic Literature Review. Proceedings of the 56th ACM Technical Symposium on Computer Science Education. 2025. DOI: 10.1145/3641554.3701863.

[20] Lai Y.-H., Lin Y.-T. Analysis of Learning Behaviors and Outcomes for Students with Different Knowledge Levels: A Case Study of Intelligent Tutoring System for Coding and Learning (ITS-CAL). Applied Sciences. 2025. DOI: 10.3390/app15041922.

[21] LearnLM Team, Eedi, Wang A. et al. AI tutoring can safely and effectively support students: An exploratory RCT in UK classrooms. arXiv preprint arXiv:2512.23633. 2025. URL: https://arxiv.org/abs/2512.23633.

[22] El Saadawi G., Tseytlin E., Legowski E., Jukic D. A natural language intelligent tutoring system for training pathologists: implementation and evaluation. Advances in Health Sciences Education. 2008. DOI: 10.1007/s10459-007-9081-3.

[23] Khan Academy. Meet Khanmigo: Khan Academy's AI-powered teaching assistant & tutor. 2024. URL: https://www.khanmigo.ai/ (дата звернення: 14.03.2026).

[24] Duolingo. Introducing Duolingo Max, a learning experience powered by GPT-4. 2023. URL: https://blog.duolingo.com/duolingo-max/ (дата звернення: 14.03.2026).

[25] Ollama. API documentation. 2024. URL: https://docs.ollama.com/api (дата звернення: 14.03.2026).

[26] Kiesler N., Lohr A., Keuning H. Exploring the Potential of Large Language Models to Generate Formative Programming Feedback. 2023 IEEE Frontiers in Education Conference (FIE). 2023. DOI: 10.1109/FIE58773.2023.10343457.

[27] Ma R., Shen Y., Koedinger K., Wu M. How to Teach Programming in the AI Era? Using LLMs as a Teachable Agent for Debugging. Lecture Notes in Computer Science. 2024. DOI: 10.1007/978-3-031-64302-6_19.

[28] Lin C.-C., Huang A. Y. Q., Lu O. H. T. Artificial intelligence in intelligent tutoring systems toward sustainable education: a systematic review. Smart Learning Environments. 2023. DOI: 10.1186/s40561-023-00260-y.

[29] Dong J., Bai Y., Xu Y., Zhou L. Large Language Models in Education: A Systematic Review. 2024 6th International Conference on Computer Science and Technologies in Education (CSTE). 2024. DOI: 10.1109/CSTE62025.2024.00031.

[30] Meta. Llama 3.1 model resources. 2024. URL: https://llama.meta.com/ (дата звернення: 14.03.2026).

ДОДАТКИ

Додаток А. Діаграми архітектури системи

У додатку стисло перелічено рисунки, які вставлено в основні розділи роботи. Вони відображають архітектуру AIHC, структуру стану сесії, послідовність сценарію та схему виконання прототипу.

Рисунок 3.1 - Компонентна архітектура AIHC для сценарію навчання базового JavaScript.

Рисунок 3.2 - Структура `UserState` та зв'язок із `SessionMemory`.

Рисунок 3.3 - Діаграма послідовності для сценарію «згенерувати вправу -> пройти тести -> дати фідбек».

Рисунок 4.1 - Схема виконання прототипу з локальною LLM, резервним режимом і `Web Worker`.

Додаток Б. Приклади промптів

Нижче наведено скорочені фрагменти промптів, які реально використовуються в прототипі. Вони демонструють обмеження на формат відповіді та заборону на повне розкриття розв'язку.

Ти виконуєш роль репетитора з базового JavaScript. Поверни лише JSON-об'єкт без пояснень. Вправа має містити: title, prompt, starterCode, functionName, concepts, rubric, tests.
Не включай повний розв'язок.

```
Ти - доброзичливий репетитор з JavaScript для початківця.
Поверни лише JSON-об'єкт.
Не давай повний розв'язок.
Врахуй attemptsCount, policy.action і перший збій у тестах.
```

Додаток В. Приклади діалогів агента

Приклад 1. Перша помилка.

Студент: Я запустив тести, але один сценарій не проходить.
Репетитор: Подивись на найпростіший приклад для порожнього масиву.
Репетитор: Не переписуй усе рішення, спочатку перевір, яке значення функція має повернути в цьому випадку.

Приклад 2. Запит на пояснення.

Студент: Поясни, чому тут потрібен цикл.
Репетитор: Цикл потрібен, коли треба пройти всі елементи масиву і поступово оновлювати результат.
Репетитор: Спробуй спершу сформулювати, яка змінна має накопичувати суму.

Додаток Г. Фрагменти програмного коду

Фрагмент 1. Явний режим пояснення у `TutorPolicy`.

```
if (forceExplanation || interactionMode === "explain") {
  return { action: "concept_explanation" };
}
if (isNaturalLanguageExplanationRequest(studentRequest)) {
  return { action: "concept_explanation" };
}
```

Фрагмент 2. Ізольоване виконання студентського коду у Web Worker.

```
const payload = { userCode, functionName, tests };
const validation = validateSandboxRequest(payload);
worker.addEventListener("error", handleWorkerError);
worker.addEventListener("messageerror", handleWorkerMessageError);
worker.postMessage(payload);
```

Фрагмент 3. Формування JSON-експорту поточної сесії.

```
return {
  topic: this.exercise?.topic ?? this.userState.currentTopic,
  difficulty: this.exercise?.difficulty ?? null,
  attemptsCount: this.userState.attemptsCount,
  firstFailure: this.firstFailure ? structuredClone(this.firstFailure) : null,
  finalRunStatus: this.lastRunResult ? { status: this.lastRunResult.status } : null
};
```